Implement simple cache with direct mapping

The assignment is to implement an 8x4 direct mapping cache with 8 cache lines (or blocks) where each line holds a block size of 4 words.

Cache line = Int(Address/4) mod 8

Cache column = Address mod 4

The CSCI6461 Simulator was modified to include a display box showing live cache. It displays the cache tag, 4-word content, and a dirty bit indicating the cache was written into. When loading a value from memory, first the cache is checked. If it's a hit, the value is fetched from cache. If it's a miss, the value is retrieved from memory and cache is updated. On a store hit, meaning the address is in the cache, the cache is updated and the dirty bit is set to 1, but main memory is not updated. This is write-back. On a store miss, the cache is updated from memory, the cache is then updated with new word that needs to be stored, and the dirty bit set to 1. This is write-allocate. If a cache block with the dirty bit set to1 is evicted, it is first updated in memory.

Code Changes

In addition to the html code change to add a table, changes were made to the javascript code within executeInstruction() for LDR (load) and STR (store). It utilizes javascript functions defined directly above in the "Cache functions" section.

Example of loading (LDR3.asm)

Memory	20	21	22	23	...	52	53	54	55
	20	30	50	150		100	0	0	0

LDR 0, 0, 20

LDR 1, 0, 22

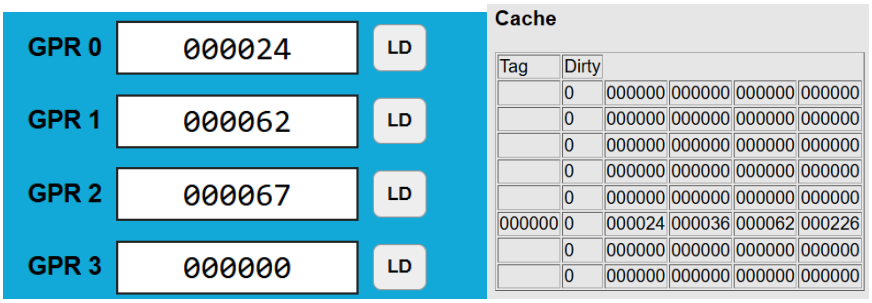
LDR 2, 0, 22

AIR 2,

LDX 1, 31

LDR 3, 1, 0

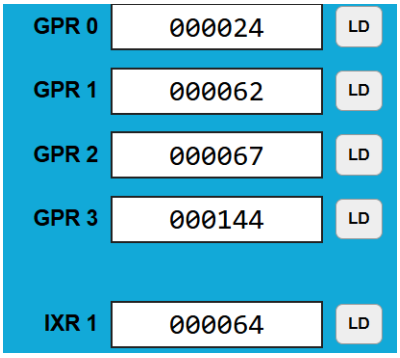
The block of memory starting at address 20 is loaded into the 6th row of the cache. 5 is added to register 2 so it's 55, but cache is not affected.



Cache

Tag	Dirty				
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
000000	0	000024	000036	000062	000226
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000

The block of memory starting at address 52 is then loaded into the 6th row of the cache which also changes the tag. The block of memory starting at address 20 is evicted.



Cache

Tag	Dirty				
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
000001	0	000144	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000

Example of storing (STR3.asm)

Memory	20	21	22	23	...	52	53	54	55
	20	0	0	0		100	0	0	0

LDR 0, 0, 20

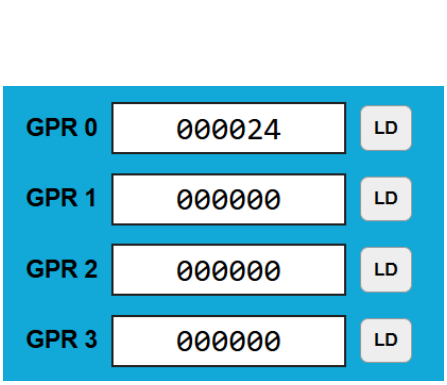
STR 0, 0, 23

LDX 1, 31

LDR 3, 1, 0

LDR 1, 0, 23

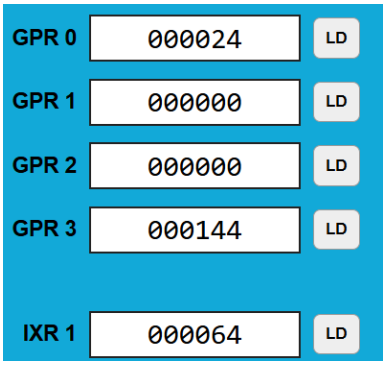
The block of memory starting at address 20 is loaded into the 6th row of the cache after loading GPR 0. The value in GPR 0, which is 20, is then stored at memory address 23. Memory is not updated; but the cache is updated in row 6 column 4, and the dirty bit is set to 1.



Cache

Tag	Dirty				
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
000000	1	000024	000000	000000	000024
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000

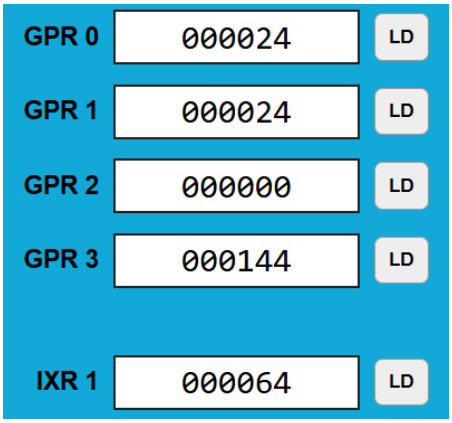
The block of memory starting at address 52 is then loaded into the 6th row of the cache which also changes the tag. The block of memory starting at address 20 is evicted. Since the dirty bit was 1, it was written to main memory upon eviction.



Cache

Tag	Dirty				
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
000001	0	000144	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000

To confirm main memory was updated upon eviction, the block of memory starting at address 20 is again loaded into the 6th row of the cache after loading GPR 1 with the contents of memory address 23. The value is now 20 instead of 0, showing main memory was updated correctly.



Cache

Tag	Dirty				
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000
000000	0	000024	000000	000000	000024
	0	000000	000000	000000	000000
	0	000000	000000	000000	000000